

Week 2 - Day 8b: Images Basics

In this exercise, we will practice several important image manipulation skills that are essential for creating, modifying, and displaying images programmatically. You will practice:

1. **Creating Images:** Initializing a new image with specified dimensions.
2. **Accessing Image Properties:** Retrieving the width and height of an image.
3. **Manipulating Pixels:** Accessing/ Setting the color values (RGB) of a specific pixel.
4. **Displaying Images:** Rendering an image on the screen/ Zooming in on smaller images to better observe pixel-level details

Image Introduction

We will be using the local `simpleimage` library for creating, manipulating and displaying images. One advantage of having a local module is that you can simply check out the source code in this the file `simpleimage.py`. Don't worry if you don't get all the syntax as it includes some concepts that we haven't taught yet. However, using the library should be straightforward. Let's dive into it.

First, we will have to load the main class `SimpleImage` from the module `simpleimage`.

```
# Run this cell to install the required dependencies otherwise the
next cell will fail
%pip install pillow

from simpleimage import SimpleImage
```

Next, we can create a blank image of a given width by height

```
# Create new image of size 40x20
image = SimpleImage.blank(4, 3)
```

The width and height are saved as attributes in the image object. We can retrieve them using the following syntax.

```
print(f"The image is of dimension {image.width}x{image.height}")
```

We can retrieve the RGB pixel values for each pixel in the image by passing the (x, y) coordinates. Note, that the indexing is - just like with lists and virtually any object in Python - starting with zero. The returned colour value for the pixel is returned as a list of three integers representing the intensity of red (R), green (G), and blue (B) respectively. The colour values are in between 0 and 255 (both including).

```
x, y = 0, 0
pix = image.get_rgb(x, y)
print(f"The pixel at location ({x}, {y}) has value {pix}")
```

The pixel value `[255, 255, 255]` means that all intensities are maximal, i.e. the pixel is white. Can you imagine of how the colour values of a red, green, and blue pixel look like? Let's try to set the pixel value of the top-left pixel `((0,0))` to red.

```
r, g, b = 255, 0, 0
image.set_rgb(x, y, r, g, b)
```

Let's try to display the image with our changed pixel value.

```
image.show()
```

Woops, our 4×3 pixel image is barely visible. Let's use the `resize_width` argument in the `show` method to display the image at a proper size.

```
image.show(resize_width=200)
```

Lovely! We can see the image with our red pixel. Congratulations! You know all the basics necessary to start working with images. □

Exercise 1: Image Basics [□]

1.1 Let's create a blank image with a white background, that is an 11 by 11 square.

```
# TODO: Your code here
size = 11
```

1.2 Now, let's set the middle pixel to a black dot by using the `set_rgb()` function.

```
# TODO: Your code here
```

1.3 After painting the center pixel of the picture black, let us paint the four corners black.

It should look something like this.

Four Dots and Middle Dot

```
# TODO: Your code here
```

1.4 Define a function called `draw_horizontal(img, x, y, length)` that will draw a horizontal black line on the given image, starting from the pixel (x, y) and extending for the specified length to the right.

```
# TODO: Your code here
def draw_horizontal(img, x, y, length):
    pass

# Use this cell to test your code
draw_horizontal(SimpleImage.blank(15, 15), 0, 7,
15).show(resize_width=200)
```

1.5 Define a function called `draw_vertical(img, x, y, length)` that draws a vertical line given image, starting from the pixel (x, y) and extending for the specified length.

```
# TODO: Your code here
def draw_vertical(img, x, y, length):
    pass

# Use this cell to test your code
draw_vertical(SimpleImage.blank(15, 15), 7, 0,
15).show(resize_width=200)
```

1.6 Define a function called `draw_line(img, x, y, length, vertical)` that draws a vertical line if the argument `vertical` is set to `True` and a horizontal line otherwise.

```
# TODO: Your code here
def draw_line(img, x, y, length, vertical):
    pass

# Use this cell to test your code
draw_line(SimpleImage.blank(15, 15), 7, 0, 15,
vertical=True).show(resize_width=200)
draw_line(SimpleImage.blank(15, 15), 0, 7, 15,
vertical=False).show(resize_width=200)
```

1.7 Define a function called `draw_diagonal(img)` that draws a diagonal line from the pixel (0,0) to (size-1, size-1). It should look something like this:

```
# TODO: Your code here

# Use this cell to test your code
draw_diagonal(SimpleImage.blank(15, 15)).show(resize_width=200)
```

1.8 Define a function `draw_triangle(img)` that draws an right-angle isocles (exactly two sides are equal length) triangle (without fill) on the image.

```
# TODO: Your code here

# Use this cell to test your code
draw_triangle(SimpleImage.blank(15, 15)).show(resize_width=200)
```

1.9 Define a function `draw_checkerboard` that draws a checker board pattern. Pixel `(0,0)` should be white, `(0,1)` should be black and so on. It should look something like this:

```
# TODO: Your code here

# Use this cell to test your code
checkerboard = draw_checkerboard(SimpleImage.blank(11, 11))
checkerboard.show(resize_width=200)
```

1.10 Write a function `invert(img)` that inverts the color of each square in the board above. i.e `pixel(0,0)` should now be black, `pixel(0,1)` white and so on. The function should only take the image as an argument and invert the color of each cell in the given image. It should look like this:

```
# TODO: Your code here

# Use this cell to test your code
invert(checkerboard).show(resize_width=200)
```

1.11 Now that you have inverted the colors of the board, let's create a function `make_red` that changes the black tiles to red tiles. It should look something like this:

```
# TODO: Your code here

# Use this cell to test your code
make_red(checkerboard).show(resize_width=200)
```

Exercise 2: Gradients [] []

2.1 Create a horizontal gradient from black to white.

Horizontal Grayscale

```
# TODO: Your code here
```

2.2 Change your code so that the gradient goes from black to red.

Horizontal Black to Red

```
# TODO: Your code here
```

2.3 Change your code so that the gradient is vertical.

Vertical Black to Red

```
# TODO: Your code here
```

Congratulations 